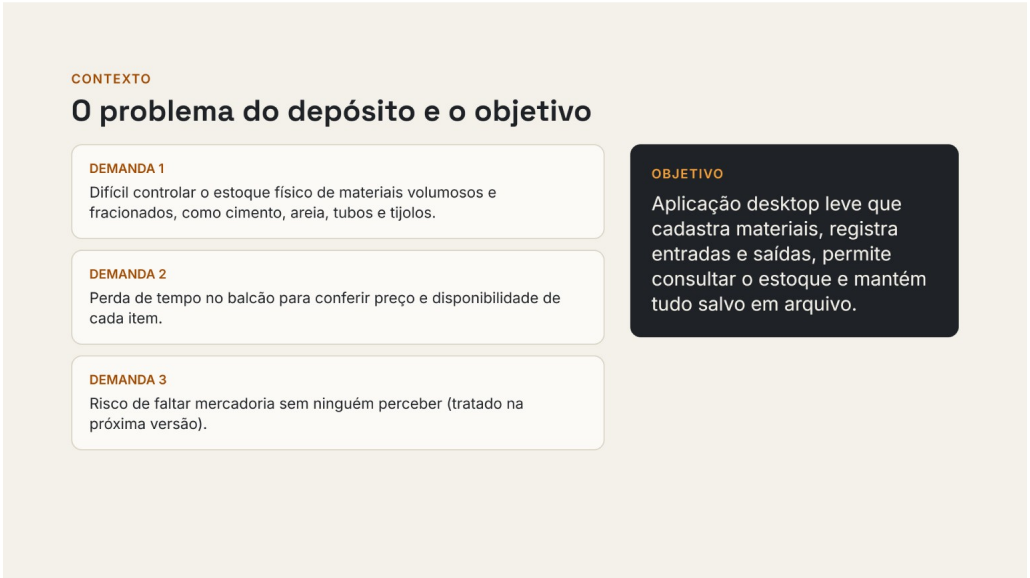


ABERTURA (cerca de 20 s). Vou apresentar o sistema de depósito de materiais de construção que desenvolvi em Python com Tkinter. Roteiro: primeiro os requisitos, histórias de usuário, casos de uso e o MVP, seguindo o capítulo 3 do livro do Valente; depois a arquitetura e os trechos principais do código. Dica: o tempo total é de cerca de 10 minutos. Os slides de código são o fim da apresentação, então se o tempo apertar, resuma os requisitos e vá direto para eles.



CONTEXTO (cerca de 40 s). Conversei sobre o dia a dia de um depósito de materiais de construção e apareceram três dores: controlar o estoque físico de itens volumosos, a lentidão no balcão para saber preço e disponibilidade e o risco de faltar mercadoria sem ninguém perceber. O objetivo do projeto é uma aplicação desktop simples que resolve o essencial: cadastrar materiais, registrar entrada e saída, consultar o estoque e guardar tudo em arquivo. Honestidade para a banca: o alerta de estoque baixo e o resumo financeiro estavam no plano inicial, mas ficaram para a próxima versão. O MVP cobre o núcleo do controle de estoque.

REQUISITOS

Requisitos funcionais: o que o sistema faz

ID	Requisito funcional	Onde está no código
RF01	Cadastrar material (ID, nome, categoria, quantidade, preço)	adicionar_produto()
RF02	Listar todos os materiais em uma tabela	atualizar_tabela()
RF03	Registrar entrada e saída (baixa) de material	controle_estoque()
RF04	Impedir saída que deixe o estoque negativo	controle_estoque()
RF05	Buscar por nome ou ID e filtrar por categoria	filtrar_tabela()
RF06	Validar o que o usuário digita (vazio, preço, quantidade)	realizar_cadastro()
RF07	Salvar e carregar os dados em arquivo	salvar_dados(), ler_arquivo()

Fonte: Valente, Engenharia de Software Moderna, cap. 3 (Requisitos). Requisito funcional = funcionalidade do sistema.

REQUISITOS FUNCIONAIS (cerca de 45 s). Segundo o Valente, requisitos funcionais descrevem o que o sistema faz, as funcionalidades. São sete: cadastrar, listar, registrar entrada e saída, impedir estoque negativo, buscar e filtrar, validar o que o usuário digita e salvar e carregar os dados em arquivo. Na terceira coluna já aponto a função do código que implementa cada um. Isso é rastreabilidade: ligar o requisito ao trecho de código, que o próprio capítulo destaca. Deixe claro o que ficou de fora: alerta de estoque mínimo e resumo do valor total.



REQUISITOS NÃO FUNCIONAIS (cerca de 30 s). Requisitos não funcionais são restrições de qualidade, e o Valente reforça que devem ser quantificados, com uma métrica. Por isso cada um tem uma meta mensurável: usabilidade, com uma saída registrada em até 1 minuto por um funcionário novo; portabilidade, sem dependências além da biblioteca padrão; confiabilidade, gravando em arquivo a cada operação; e desempenho, com atualização da tabela em até 1 segundo. Atenção: as metas de usabilidade e desempenho são metas de projeto, ainda não medidas formalmente. Se perguntarem, diga isso.



HISTÓRIAS DE USUÁRIO (cerca de 50 s). No capítulo 3, uma história de usuário é cartão, conversas e confirmação. O cartão é a frase no formato "Como um papel, eu gostaria de...". A confirmação é o teste de aceitação. Tenho quatro histórias do MVP, cada uma com seu critério de confirmação: HU1, buscar e filtrar; HU2, cadastrar; HU3, registrar entrada; HU4, dar baixa sem deixar o estoque negativo. Duas histórias ficaram no backlog: alerta de estoque baixo para o estoquista e valor total em estoque para o proprietário. Ponto de INVEST: cada história é pequena, testável e valiosa para quem a pediu.

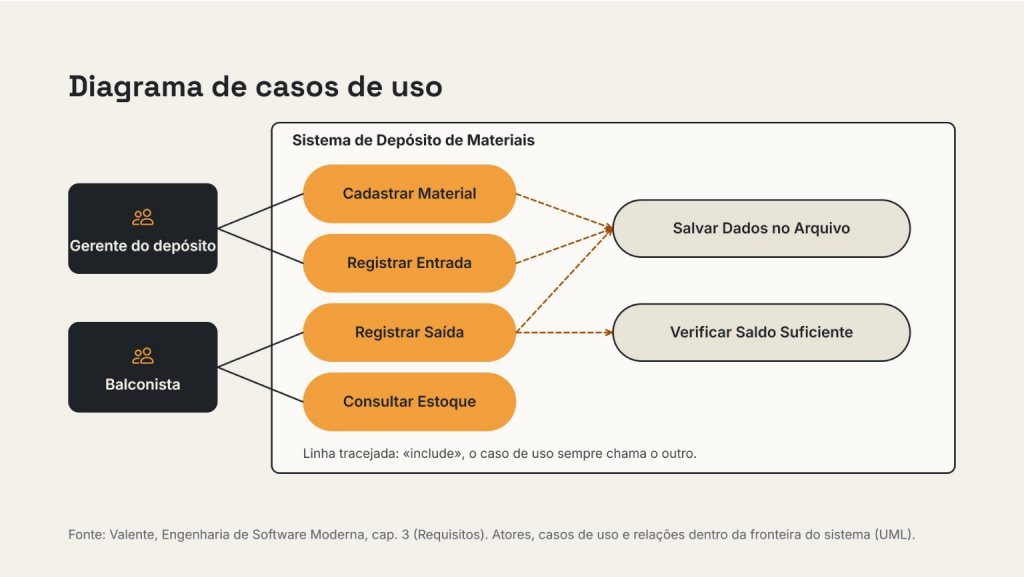
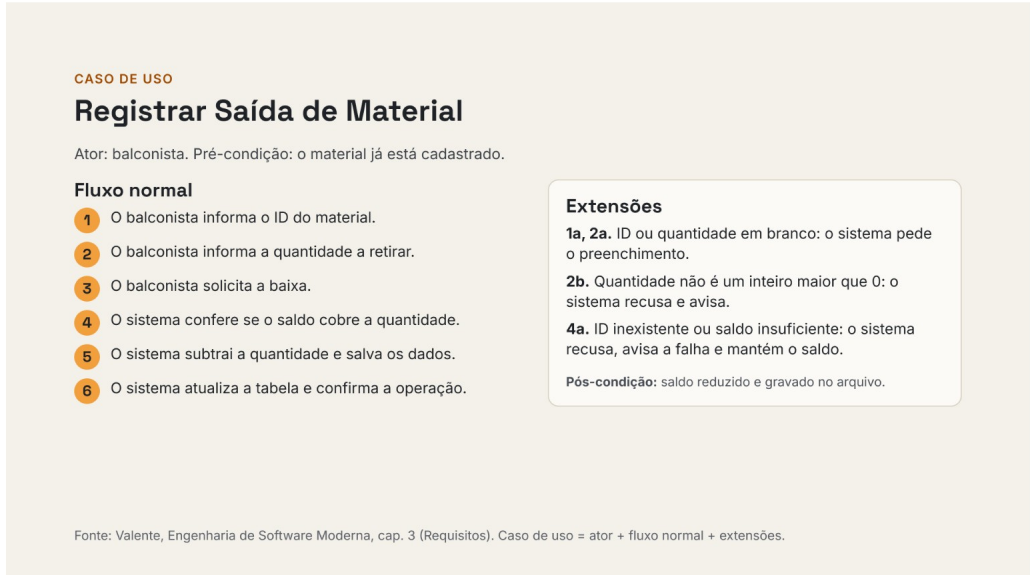
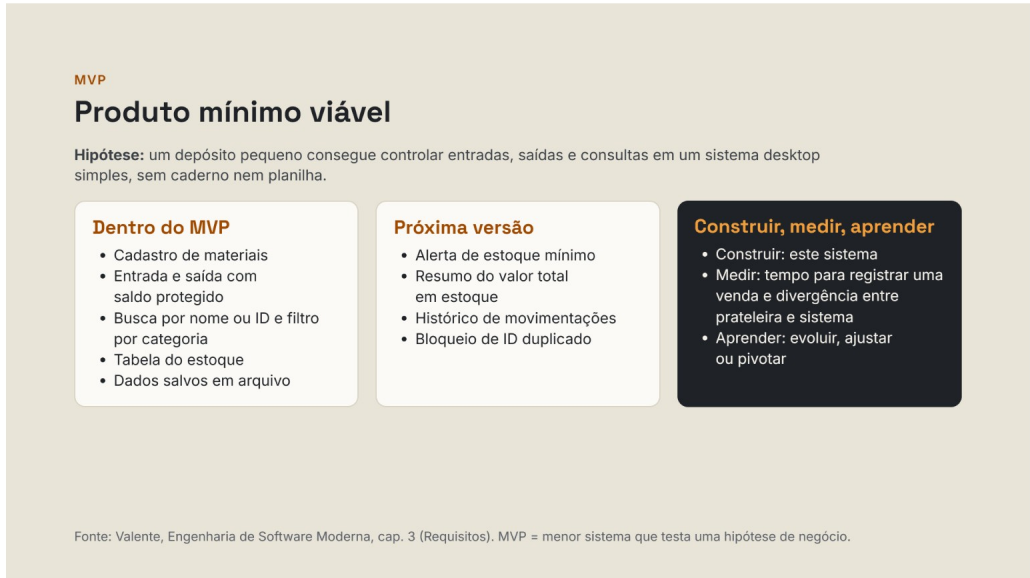


DIAGRAMA DE CASOS DE USO (cerca de 50 s). Este é o índice gráfico dos casos de uso, no padrão UML: os atores ficam fora da fronteira do sistema e os casos de uso dentro dela. Dois atores: o gerente do depósito, que cadastra material e registra entradas, e o balconista, que registra saídas e consulta o estoque. As linhas tracejadas são relações de inclusão: cadastrar, dar entrada e dar saída sempre chamam “Salvar Dados no Arquivo”, e a saída também chama “Verificar Saldo Suficiente”, que é o requisito RF04. Na próxima tela detalho um deles em texto.



CASO DE USO EM TEXTO (cerca de 40 s). No Valente, um caso de uso tem nome com verbo no infinitivo, um ator, o fluxo normal (o caminho feliz) e as extensões, que são os desvios e erros numerados com letras. Escolhi “Registrar Saída de Material” porque é o caso mais importante do estoque. O fluxo normal tem seis passos, abaixo do limite de nove sugerido por Cockburn. As extensões saem direto do código: campos em branco, quantidade que não é inteiro positivo e ID inexistente ou saldo insuficiente. Esta última é o requisito RF04. O passo 5 é a inclusão de “Salvar Dados no Arquivo” do diagrama.



MVP (cerca de 50 s). O Valente define MVP como o sistema funcional com o conjunto mínimo de funcionalidades para testar uma hipótese. A minha hipótese: um depósito pequeno consegue controlar entradas, saídas e consultas em um sistema desktop simples. Dentro do MVP ficou o núcleo: cadastro, entrada e saída com saldo protegido, busca e filtro, tabela e persistência em arquivo. Ficou para depois: alerta de estoque mínimo, resumo do valor em estoque, histórico de movimentações e bloqueio de ID duplicado. Na terceira coluna, o ciclo construir, medir e aprender, com métricas acionáveis, como o tempo para registrar uma venda, e não métricas de vaidade. Esse resultado decide se evolui, ajusta ou pivota.



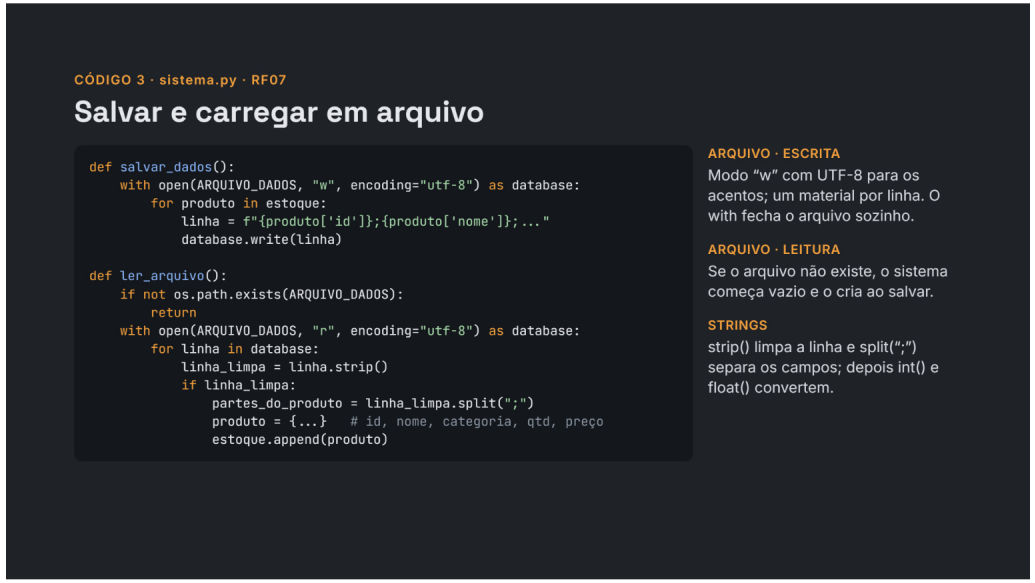
ARQUITETURA (cerca de 30 s). O projeto tem três arquivos. interface.py cuida só da tela. sistema.py concentra as regras do estoque. dados_estoque.txt guarda os dados, um material por linha, com os campos separados por ponto e vírgula, como neste exemplo. A tela chama as funções de sistema.py sempre que precisa alterar o estoque; assim as regras ficam num lugar só e não se misturam com os botões. Agora os trechos principais do código, ligados aos requisitos que vimos.



CÓDIGO 1: MODELO DE DADOS E CADASTRO (cerca de 50 s). Trecho resumido de sistema.py, requisito RF01. Aqui estão as estruturas de dados do projeto. CATEGORIAS é uma tupla, porque a lista de categorias é fixa. O estoque é uma lista, e cada material é um dicionário com id, nome, categoria, quantidade e preço. A função adicionar_produto usa um if para validar: preço zero ou negativo e quantidade negativa são recusados e a função devolve False. Depois, strip() tira espaços e title() deixa o nome com iniciais maiúsculas antes de guardar. Se perguntarem: hoje o código não bloqueia ID repetido. Isso está na lista de próximas versões.



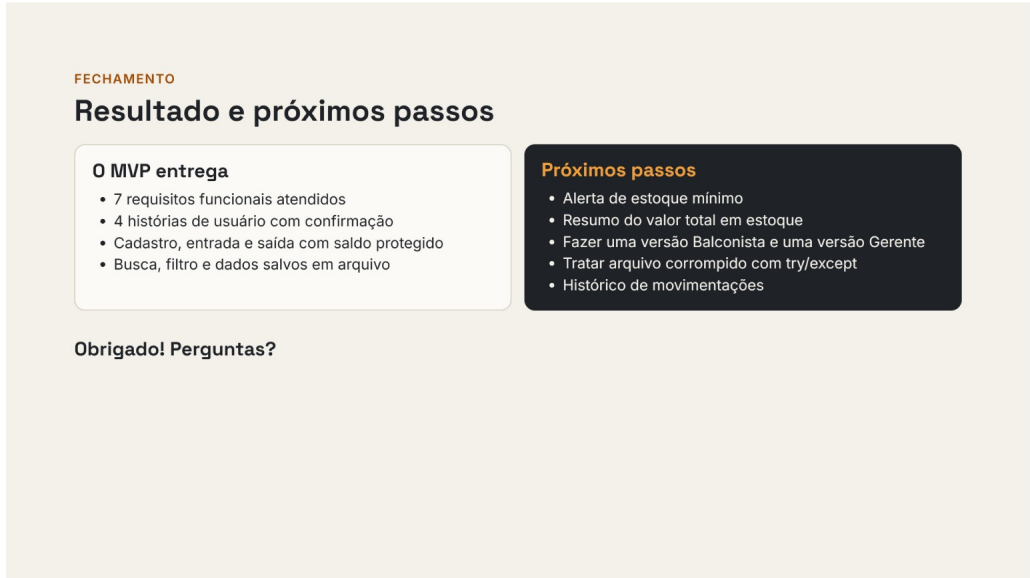
CÓDIGO 2: CONTROLE DE ESTOQUE (cerca de 50 s). Trecho de sistema.py, requisitos RF03 e RF04. Esta é a regra central do sistema. Uma única função trata entrada e saída: recebe o ID e uma quantidade positiva (entrada) ou negativa (saída). O for percorre a lista até achar o produto. Calcula o novo saldo e, com if e else, decide: se ficaria negativo, recusa e não muda nada, que é o RF04; senão atualiza, chama salvar_dados() e devolve True. Se o ID não existe, o fim da função avisa e devolve False. A interface usa esse True ou False para mostrar sucesso ou erro.



CÓDIGO 3: PERSISTÊNCIA (cerca de 50 s). Trecho resumido de sistema.py, requisito RF07 e RNF de confiabilidade. salvar_dados abre o arquivo em modo escrita com UTF-8, para manter os acentos, e grava cada material em uma linha, campos separados por ponto e vírgula. O with garante que o arquivo é fechado. ler_arquivo faz o caminho inverso: se o arquivo não existe, o sistema começa vazio; se existe, lê linha por linha, limpa com strip, separa com split e monta o dicionário, convertendo quantidade para int e preço para float. O salvamento é chamado a cada cadastro e a cada movimentação, então os dados sobrevivem ao fechar o programa. Se perguntarem: o plano previa JSON, mas a versão final usa TXT. A leitura ainda não trata arquivo corrompido, e isso está nas próximas versões.



CÓDIGO 4: INTERFACE (cerca de 50 s). Trecho resumido de interface.py, requisitos RF03 e RF06. Os botões Entrada e Baixa chamam dar_entrada e dar_saida, e as duas usam movimentar. Ela lê os campos, valida (vazio, número inteiro, quantidade maior que zero, usando try e except para não travar com texto) e multiplica por 1 ou por -1, conforme entrada ou saída. Depois chama controle_estoque, a função do slide anterior, e usa o retorno: se deu certo, atualiza a tabela (que é um Treeview, preenchido por um for sobre a lista de materiais), limpa os campos e mostra a mensagem em verde; se não, mostra o erro em vermelho. A tela só conversa com as regras por essa função, que é a separação de responsabilidades do slide de arquitetura.



FECHAMENTO (cerca de 40 s). O MVP entrega o núcleo do controle de estoque: sete requisitos funcionais, quatro histórias com confirmação, movimentação com saldo protegido, busca, filtro e persistência em arquivo. Os próximos passos vêm das limitações que já identifiquei: alerta de estoque mínimo, resumo do valor total, bloqueio de ID duplicado, tratamento de arquivo corrompido e histórico de movimentações. Obrigado, e fico à disposição para perguntas.